



Database Management Systems

Database Design

Anchor: Dr. Nagasundari S, Dept of CSE

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Special Acknowledgement to all CSE Professors who contributed to this endeavor

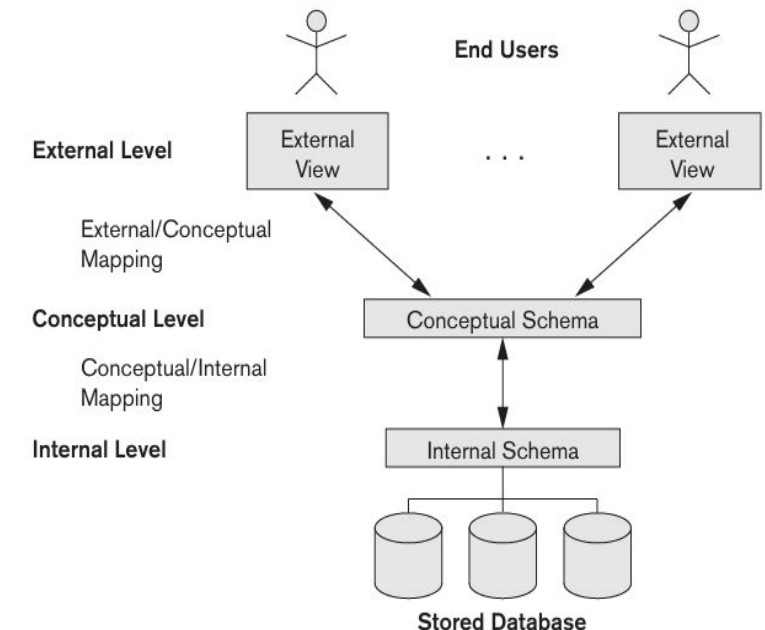
Dr. Nagasundari S

Department of Computer Science and Engineering



- DBMS plays an important role in many areas. One of the main requirements is that it **must retrieve the data efficiently**.
- This need for efficiency has led the database developers to use complex data structures to represent data in a database
- Many database-system users are **not computer trained** and do not know how to write complex queries for getting information they are looking for.
- Hence developer hide all these complexity from the users through several layers of data abstraction, in order to simplify the users' interaction with the system
- The different levels of data abstraction:
 - Physical Level
 - Logical level
 - View level

Figure 2.2
The three-schema architecture.



Let us now have a look at what happens at each level:

Physical Level :

- It is the lowest level of data abstraction
- It describes how the data is actually stored in the computer

Logical Level :

- This describes what kind of data is stored in the database and how different pieces of information are related to each other
- The logical level thus describes the entire database in terms of a small number of relatively simple structures.
- This level provides a more organized and simpler view of the data

Let us now have a look at what happens at each level:

View Level :

- This is the highest level of abstraction that individual users interact with and that only displays a part of the entire database
- Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database.
- At this level, different users might see different parts of the database, depending on their needs and permissions

Now we can notice that:

- Although the implementation of simple structures at the logical level may involve complex physical-level structures, the user at the logical level does not need to be aware of this.
- Similarly the user who views the database for various purposes need not be aware of the Logical level implementation, for them it would be like an interface and need not worry about the logical view

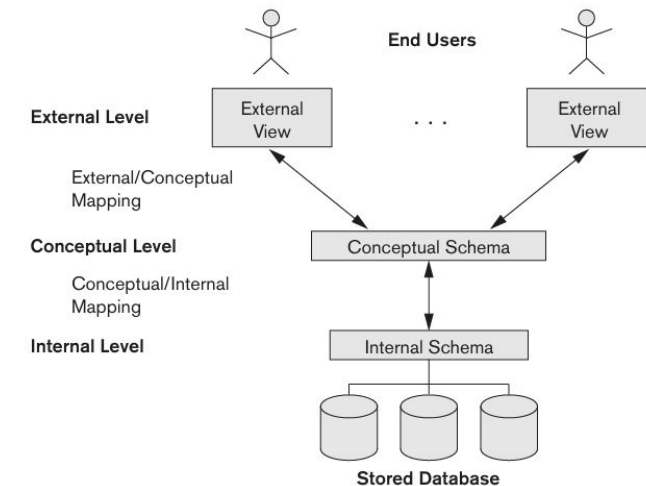
PHYSICAL DATA INDEPENDENCE:

- It refers to the characteristic of being able to modify the physical schema without any alterations to the conceptual or logical schema, done for optimization purposes.

Example :

- the Conceptual structure of the database would not be affected by any change in the storage size of the database system server. Changing from **sequential to random access** files is one such example.
- These alterations or modifications to the physical structure may include:
 - Utilizing new storage devices.
 - Modifying data structures used for storage.
 - Altering indexes or using alternative file organization techniques etc.

Figure 2.2
The three-schema architecture.

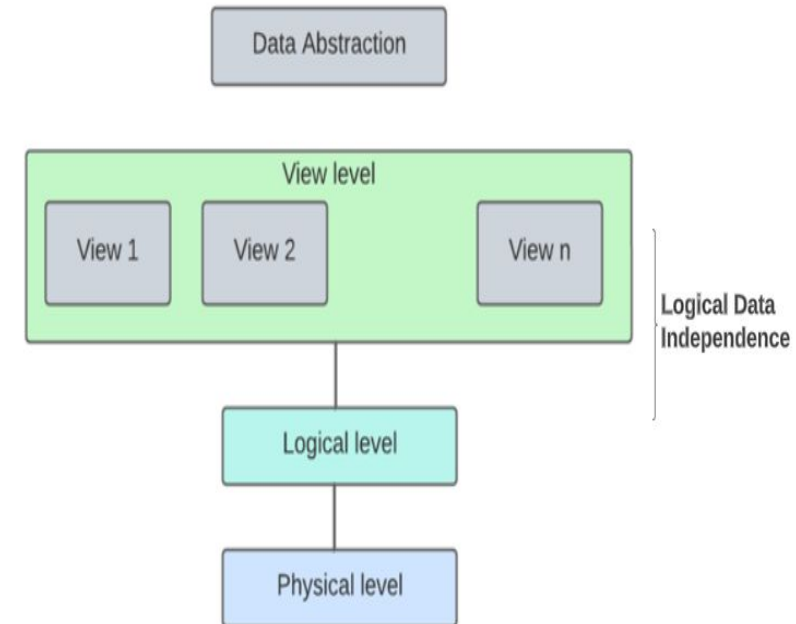


LOGICAL DATA INDEPENDENCE

- It refers characteristic of being able to modify the logical schema without affecting the external schema or application program. The user view of the data would not be affected by any changes to the conceptual view of the data.

Example:

- These changes may include the insertion or deletion of attributes,
- altering table structures entities or relationships to the logical schema, etc.



Now let us understand this with the help of an example of a University database

- Let the university have the given Record types/ tables as shown

Now for the given database,

- **Physical level:**
 - Specify how student records, course information, and department data are **stored on the hard drive or in memory.**
- **Logical Level:**
 - It would define entities like "Department," "Student," and "Course" and their attributes (e.g., department name, student ID, course code).
 - It also outlines the relationships between these entities, such as "Students belong to Departments" and "Courses are taken by Students."

Department

Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------

Now let us understand this with the help of an example of a University database

- Let the university have the given Record types/ tables as shown

Now for the given database,

- **View Level:**
 - A faculty member may have access to a view showing only the courses they are teaching and the students enrolled in those courses.
 - A student, on the other hand, may have a view displaying only the courses they are registered for and their respective grades.
 - These views provide a simplified and customized perspective of the database for different users.

Similarly , how could physical and logical data independence be of an advantage in the case of the university database?

Department

Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------

Let us consider the situation where the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance. What would be the consequence of this?

Without Physical data independence:

- Minor changes to the storage system or hardware upgrades could potentially break existing queries, applications, and interfaces.
- As a result, any update to the storage system would require extensive modifications to the application layer, leading to higher maintenance costs and longer development cycles.

Department

Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------

Let us consider the situation where the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance. What would be the consequence of this?

With physical data independence:

- the database administrators can make these changes without altering the logical schema, which means users' queries and applications will continue to work seamlessly.
- It allows for easier maintenance, upgrades, and performance enhancements without disrupting the application layer.

Department

Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------

Let's say the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?

Without Logical data independence:

- Any alteration to the logical schema, such as adding or removing attributes, or changing relationships between entities, would result in changes propagating throughout the applications that rely on the database.
- This would force developers to modify all affected application code to accommodate the changes in the schema.
- Consequently, even minor changes to the data model could lead to widespread disruption and downtime for the applications, increasing the risk of introducing errors and inconsistencies.

Department

Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------

Let's say the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?

With Logical data independence:

- The database designers can make these modifications without impacting the existing application programs that access the database.
- This flexibility allows for easier adaptability to changes in the University's requirements and business rules without causing application disruptions.

Department

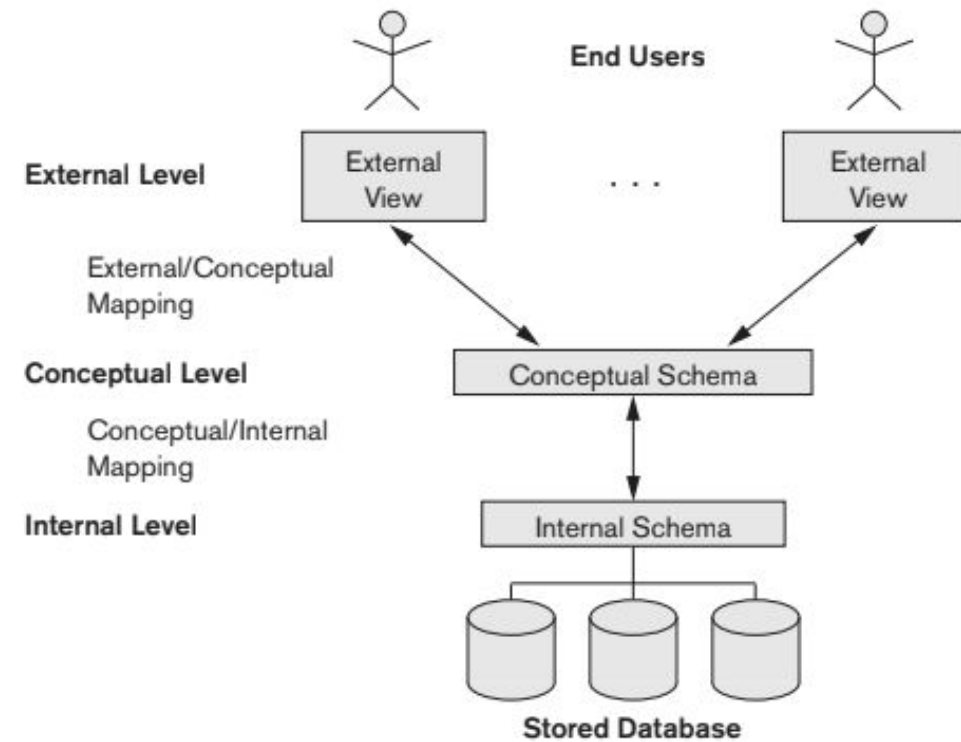
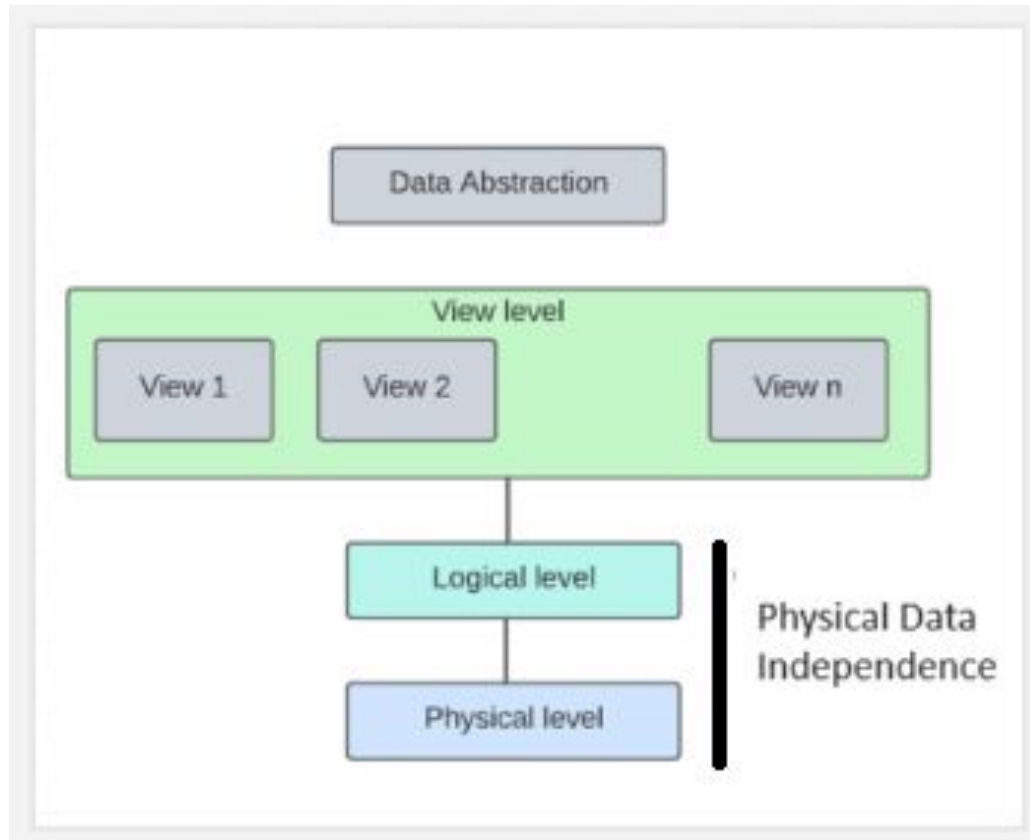
Dept_ID	Dept_Name	Building
---------	-----------	----------

Course

Course_ID	Course_Name	Credits	Dept_ID
-----------	-------------	---------	---------

Student

SRN	Name	Dept_ID	Total_Credits
-----	------	---------	---------------



- Databases change over time as information is inserted and deleted or modified.
- **The collection of information stored in the database at a particular moment is called an instance of the database.**
- **The overall design of the database is called the database schema.**
- Let us try to understand what that exactly means with an analogy to a program written in any programming language
 - **Database Schema** => variable declarations along with the associated type definitions
 - **Instances** => Value of the variable at a particular point in time in the program

- The actual data in a database may change quite frequently.
- The data in the database at a particular moment in time is called a **database state** or **snapshot**.
- It is also called the *current set* of **occurrences** or **instances** in the database.
- In a given database state, each schema construct has its own current set of instances.
- Many database states can be constructed to correspond to a particular database schema.

- The distinction between database schema and database state is very important.
- When we **define** a new database, we specify its database schema only to the DBMS.
- At this point, the corresponding database state is the empty state with no data.
- We get the initial state of the database when the database is first **populated** or **loaded** with the initial data.
- From then on, every time an update operation is applied to the database, we get another database state. At any point in time, the database has a *current state*.

A simplified COMPANY relational database schema

EMPLOYEE

F.K.

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------

P.K.

DEPARTMENT

F.K.

Dname	<u>Dnumber</u>	Dmgr_ssn
-------	----------------	----------

P.K.

Sample database state for the relational database schema

EMPLOYEE

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

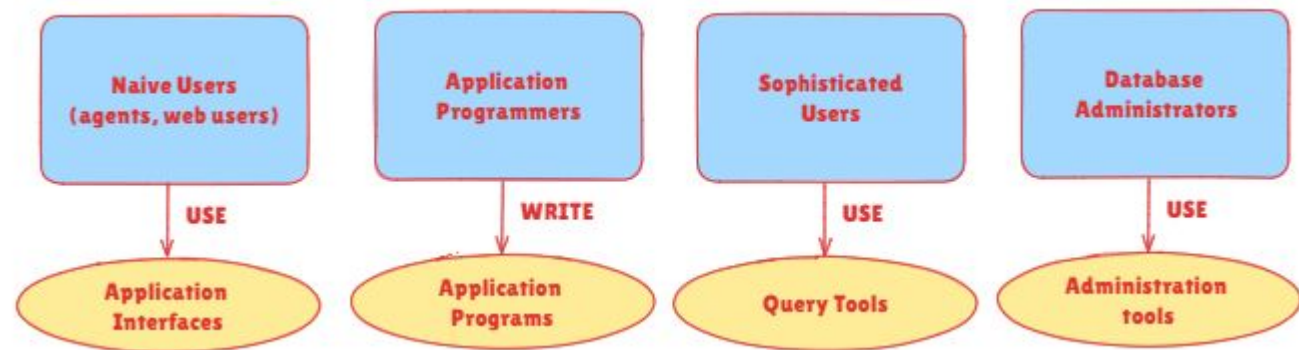
People who work with a database can be categorized as:

1. Database Users
2. Database Administrators.

Database Users

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for different types of users.

1. Naïve Users
2. Application Programmers
3. Sophisticated Users
4. Database Administrators



Naïve Users:

- Naive users also known as **Parametric End users**, don't have any knowledge of databases but still frequently use the database applications to get the desired results. With the help of the interface provided by the DBMS applications.
- **For Example:** Have you ever tried to book a ticket for your favorite movie? It's straightforward and easy.
- But, do you have any idea about the complex database operations happening behind the scenes to make this process smooth and efficient?
- In database terminology, you are a **naïve user** or a **parametric end user**. This means you interact with the database through a predefined interface without needing to know anything about how the database works.



Application programmers :

- Application Programmers also known as Back-End Developers, are computer professional users who are responsible **for developing the application programs.**
- **Example:** For an e-commerce site, application programmers connect the app to the database, write queries to display products, manage user registrations and orders, handle database errors, and secure user data.

Sophisticated Users:

- Sophisticated users are individuals who possess a deep understanding of database management systems (DBMS) and are proficient in using Data Definition Language (DDL) and Data Manipulation Language (DML) commands.
- **Example:** Data engineers and developers often access databases directly using SQL queries to fetch, delete, update, or insert data, making them sophisticated users.

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data.

Database Administrator is an individual or a team of users who defines the database schema and takes charge of controlling various levels of the database within the organization.

Functions of a Database Administrator (DBA):

- 1. Schema definition:** DBA creates the original database schema using data definition statements (DDL).
- 2. Storage structure and access-method definition:** DBA specifies parameters for data physical organization and index creation.
- 3. Schema and physical-organization modification:** DBA makes changes to reflect organization's needs or enhance performance.
- 4. Granting authorization for data access:** DBA regulates user access by granting different types of authorization.
- 5. Routine maintenance:** DBA performs periodic backups, ensures disk space availability, and monitors database performance.

Database Management Systems

Typical DBMS Component Modules

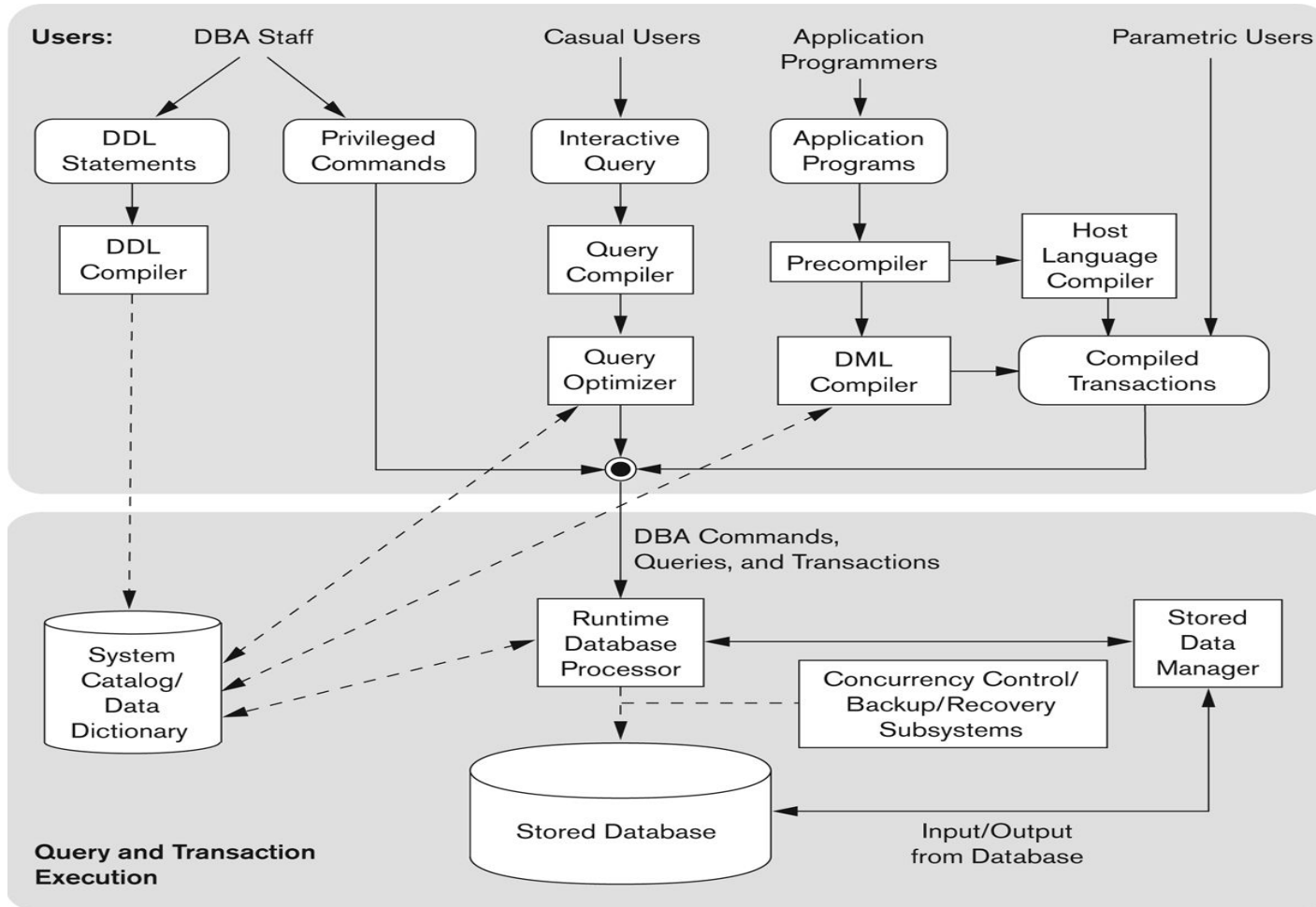


Figure 2.3
Component modules of a DBMS and their interactions.

- SDM - controls access to DBMS info that is stored on disk (dotted lines)
- DDL Compiler - processes schema definitions specified in the DDL and stores metadata in catalog
- Query Compiler – handles high level queries - parses and compiles a query and calls to runtime processor for execution
- Pre Compiler – extracts DML commands from application program
- Run time db processor – executes privileged commands/queries/canned transactions.

- Loading Utility – Load existing data files into a database. Includes data conversion tools from old format to new database format
- Backup Utility - Back up of the database periodically (Incremental/Total)
- Database Storage Reorganization - Reorganizing database file structures and create new access paths to improve performance.
- Performance Monitoring Utilities – Monitors database usage and provides statistics to DBA

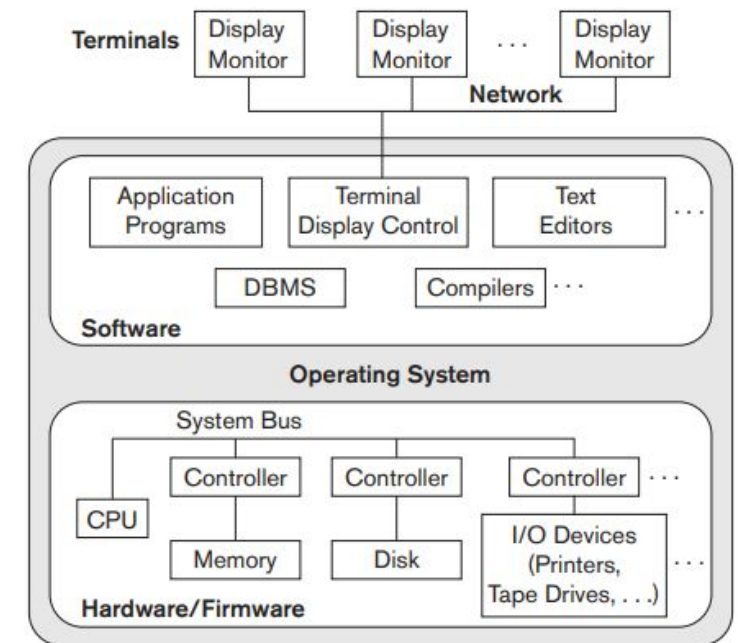
- Centralized databases (Centralized architecture)

Characteristics:

- **Single Location:** All data is stored and managed on a central server or database.
- **Shared Memory:** Suitable for systems with multiple CPUs that access a common shared memory space.
- **Resource Allocation:** Resources like CPU, memory, and storage are centralized, making it easier to manage and maintain.

Use Cases:

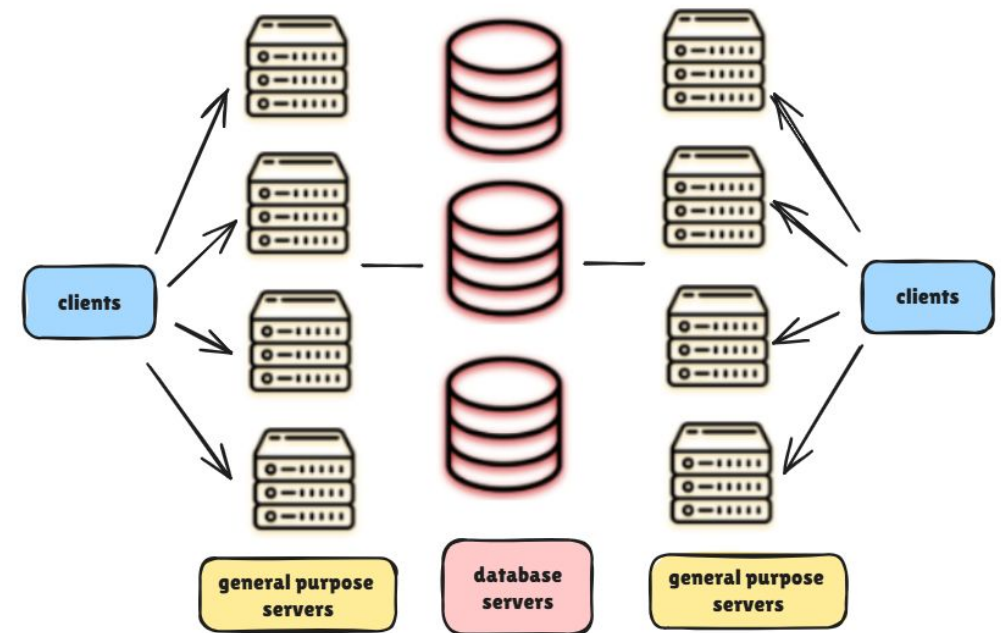
- **Small to Medium-Sized Applications:** Suitable for applications with moderate data needs and user loads.
- **Legacy Systems:** Often used in older systems where scaling is not as critical or feasible.



Centralized/Shared Memory DB

Client Server Architecture:

- Client-server architecture is a model where a client (the user or application) interacts with a server (the database system) to access and manage data.
- The client is the user-facing component that interacts with the server to request data or perform operations. (web browser, mobile apps, desktop applications)
- The server is the backend component that manages the database, processes requests from clients, and handles data storage and retrieval. (MySQL, Oracle, Postgres)



Parallel databases:

- Designed to run on a cluster of multiple machines
- Enables better scalability and higher processing capabilities.
- Many cores shared memory and Shared Disk

Parallel databases are ideal for **high-performance applications** requiring **extensive data processing**, such as large-scale data warehousing, real-time analytics, and **high-throughput** transactional systems.

Example: How do you think NASA manages and processes the vast amounts of satellite data it collects?

Answer: NASA uses parallel databases and high-performance computing (**HPC**) systems. NASA can efficiently handle and analyze massive datasets from satellite missions, enabling quick and insightful analysis for complex tasks like climate modeling and environmental monitoring.



NASA's HPC Cluster

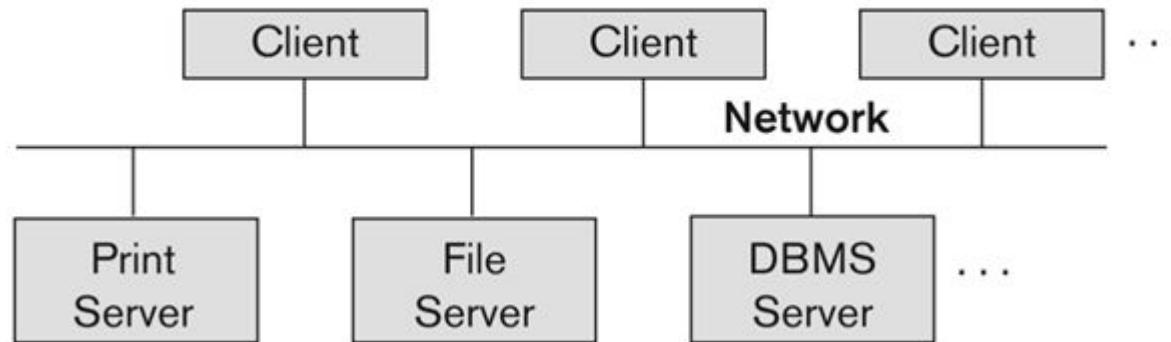
Distributed Database:

A distributed database is a database system in which **data is stored across multiple physical locations**. These locations can be spread across various machines within a single data center or across different geographic locations.

Advantages:

- **Scalability:** Can handle large amounts of data and high traffic by distributing the load across multiple nodes.
- **Fault Tolerance:** Provides high availability and fault tolerance through data replication and distribution.
- **Performance:** Can improve performance by distributing queries and data access across multiple servers.





In a two-tier database architecture, there are two main components: the client (**user interface**) and the server (**database**).

- **Client**: This is the user interface where users interact with the application. It can be a desktop application or a web-based interface.
- **Server**: This is where the database is stored and managed. The server handles data storage, retrieval, and management tasks.
- Client programs send query and transaction requests via ODBC or JDBC API, the server processes these requests and sends the results back to the client, which then processes and displays the results.

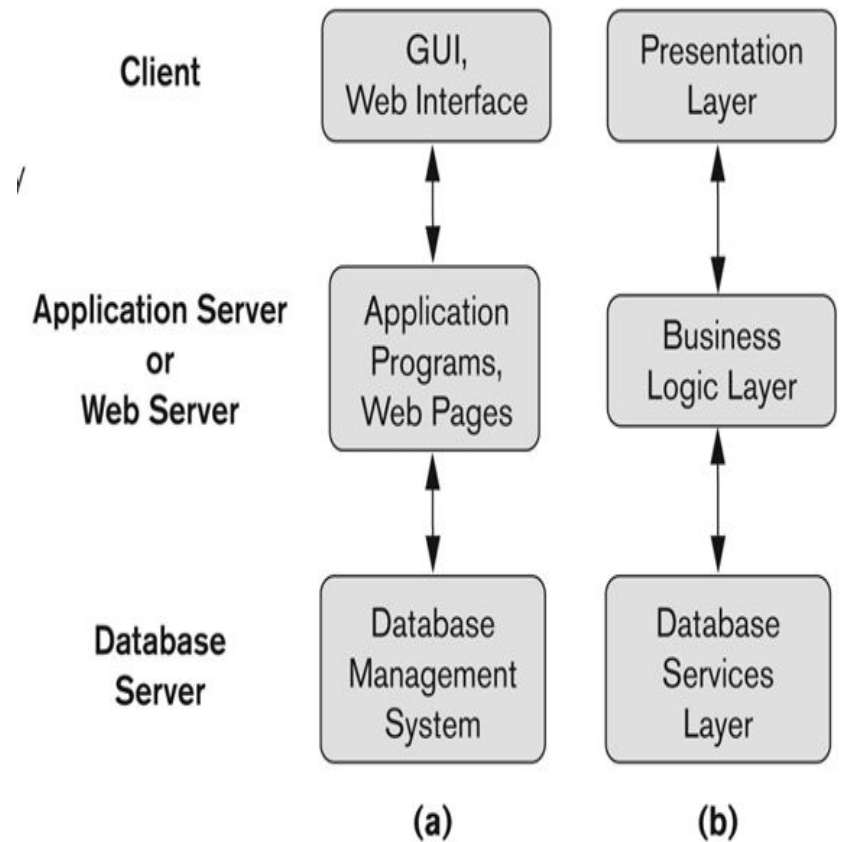
Clients

- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.

Servers

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
 - > ODBC: Open Database Connectivity standard
 - > JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
 - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
 - Acts like a conduit for sending partially processed data between the database server and the client.
- **Three-tier Architecture Can Enhance Security:**
 - **Database server only accessible via middle tier**
 - **Clients cannot directly access database server**



1. "Application programmers are responsible for defining storage structures and access methods in the database." – True/False?
2. "A _____ user interacts with a system using predefined interfaces and does not need to write code."
3. "The role of a _____ includes creating the original database schema and managing user access."
4. "What type of user would most likely use a query language to retrieve specific data from a database?"

1. "Application programmers are responsible for defining storage structures and access methods in the database." – **False (Database Administrators handle this)**
2. "A Naïve user interacts with a system using predefined interfaces and does not need to write code."
3. "The role of a Database Administrator includes creating the original database schema and managing user access."
4. "What type of user would most likely use a query language to retrieve specific data from a database?"

Sophisticated User

Some questions?

- What are the four main phases of database design? Briefly explain what happens in each phase.
- What is the role of the **storage manager** in a DBMS? List any three components it manages.
- What are the three main steps in **query processing**, and why is query optimization important?
- Define a transaction in DBMS. What are the key properties a transaction must maintain?
- Define the three levels of data abstraction in a DBMS and explain with an example from a university database.
- What is the difference between physical and logical data independence? Give one example for each.
- Differentiate between database schema and database instance with an analogy from programming.
- What is a view schema? How does it help with security and user customization?



THANK YOU

Anchor: Dr. Nagasundari S, Dept of CSE

nagasundaris@pes.edu